SUKP – work in progress

DQN-Guided Local Search for the Set-Union Knapsack Problem

Dung Truong

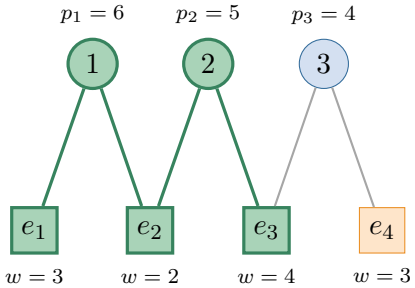
I. THE SET-UNION KNAPSACK PROBLEM

A. Problem definition

- m items with profits p_i, n elements with weights w_j, capacity C.
- Item i covers a set of elements U_i. An element shared by several chosen items is paid *once*.

$$\max_{S} \; P(S) = \sum_{i \in S} p_i \quad \text{s.t.} \quad W(S) = \sum_{j \in \bigcup_{i \in S} U_i} w_j \leq C$$

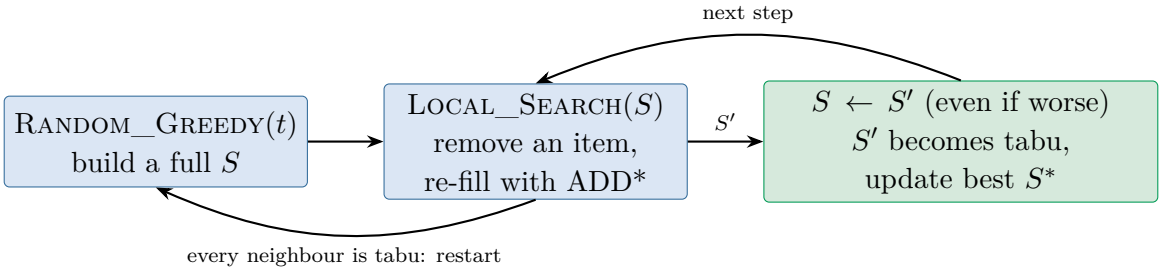
- NP-hard: the 0-1 knapsack problem is the case where no two items share an element [1, 2].
- Benchmark: 60 instances. Set I: 30 with 85–500 items/elements [3]; Set II: 30 with 585–1000 [4].
- Density $\alpha \in \{0.10, 0.15\}$ of the item-element relation; capacity $C = \beta \sum_j w_j$, $\beta \in \{0.75, 0.85\}$.



$C = 9$. Optimal $S = \{1, 2\}$ covers e_1, e_2, e_3:
$W = 3 + 2 + 4 = 9 \leq C$, $P = 11$.
Paying per item would cost $5 + 6 = 11 > C$.

II. OUR SOLVER

A. Main loop



- One step: try removing each of the r_{num} lowest-ratio items, let ADD* re-fill the knapsack, keep the best non-tabu result S'.
- Move to S' even when it is worse than S, to escape local optima.
- Tabu list: every accepted solution, stored as a 64-bit hash, so the search never cycles.
- Parameters as in the paper: $r_{num} = a_{num} = 2$, $t = \sqrt{\max(m, n)}$. Stops on a time or step budget.

[5] W. Zhu and L. Luo (2022)

II. OUR SOLVER

B. Value-weight ratio and greedy construction

$$AW(S,i) = \sum_{j \in U_i,\ c_j = 0} w_j, \qquad R(S,i) = \frac{p_i}{AW(S,i)}$$

- c_j: number of chosen items covering element j.
- AW: weight that adding i newly covers. For $i \in S$, the weight only i covers ($c_j = 1$).
- *Free* items ($AW = 0$) raise the profit at no cost: always added.
- High R: add or keep. Low R: remove first.

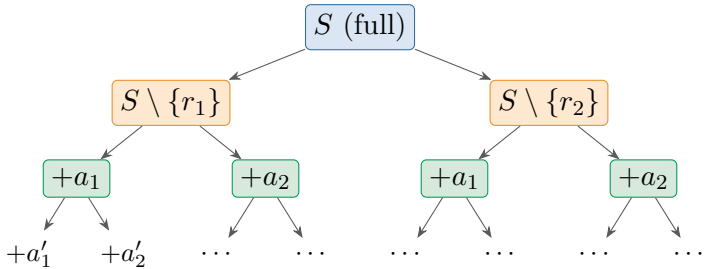
Random_Greedy(t)
repeat: add all free items;
  sample t feasible candidates,
  add the one with the highest R
until no item fits

Incremental state (C++)
keep c_j for every element: adding or removing item i costs $O(|U_i|)$, and W changes only when some c_j goes $0 \leftrightarrow 1$.

[5] W. Zhu and L. Luo (2022)

II. OUR SOLVER

C. Local search and ADD*



r_1, r_2: the two lowest-ratio items of S. a_1, a_2: the two best-ranked candidates at each node. A branch ends when nothing more fits.

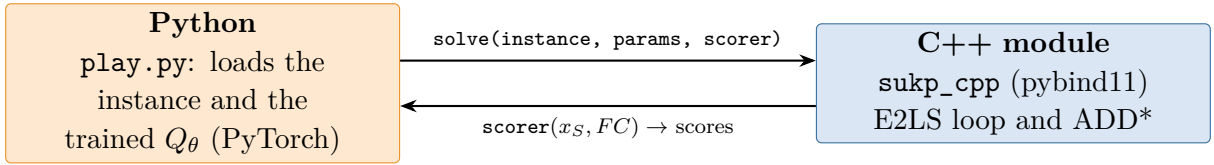
- **Remove**: each of the $r_{num} = 2$ lowest-R items, one at a time.
- **ADD***: add the free items, rank the feasible candidates FC, branch on the best $a_{num} = 2$, recurse.
- Tabu solutions are skipped at every node.
- Result: the best non-tabu solution over both trees.

Ranking FC is where learning plugs in.
Paper: by $R(S, i)$. Ours: by the DQN's $Q_\theta(s, \text{add } i)$ (Section III).

[5] W. Zhu and L. Luo (2022)

II. OUR SOLVER

D. Implementation and Python binding

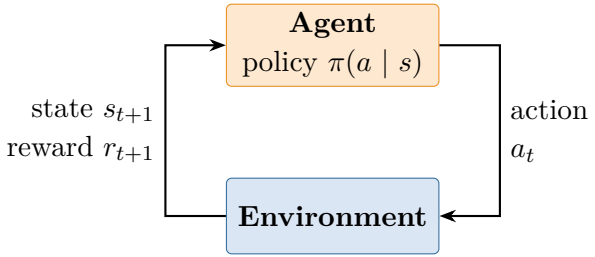


- C++17, `-O3`. The same search runs with the paper rule or with the network.
- The network is called only at ADD* nodes with more than a_{num} candidates.
- Speed with the paper rule (one core): about 14,000 steps/s at $m = 100$ and 240 steps/s at $m = 1000$.
- Every returned solution is re-checked in Python.

```
# ADD* as in the paper
python play.py --ranker ratio
# ADD* ranked by the pretrained DQN
python play.py --ranker dqn
```

III. ADD* RANKED BY A PRETRAINED DQN

A. Reinforcement learning in a nutshell



In our SUKP setting
state s: the current solution x_S
action a: add or remove one item
reward r: gain in the best profit

- Goal: a policy that maximises the expected return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$, with discount $\gamma \in [0, 1)$.
- Action value $Q^\pi(s, a) = \mathbb{E}_\pi[\, G_t \mid s_t = s,\ a_t = a \,]$: how good action a is in state s.

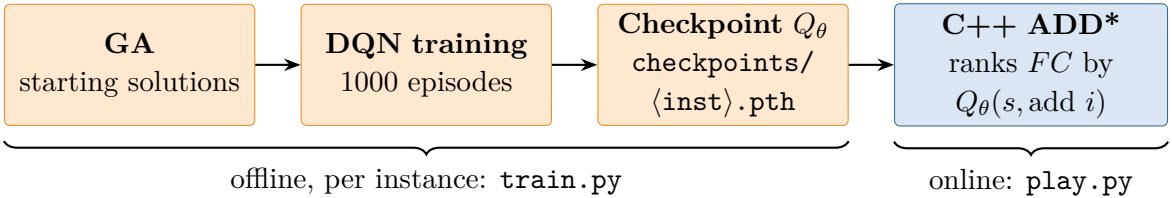
$$Q^*(s, a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^*(s', a') \,\big]$$

- Q-learning moves $Q(s, a)$ toward $r + \gamma \max_{a'} Q(s', a')$ after every step, then acts by $\arg\max_a Q(s, a)$.
- Exploration: ε-greedy tries a random action with probability ε.
- Too many states for a table (2^m solutions), so Q is approximated by a neural network: **DQN**.

[6] R. S. Sutton and A. G. Barto (2018); [7] C. J. C. H. Watkins and P. Dayan (1992)

III. ADD* RANKED BY A PRETRAINED DQN

B. Training first, then search



- **Training first**: on the instance, the DQN learns which single add/remove move improves a solution.
- **Then search**: E2LS runs unchanged except for how ADD* ranks its candidates.
- The gap input is set to 0 at ADD* nodes, since ADD* only adds items.

paper: $\text{rank}(i) = R(S,i) = p_i / AW(S,i)$

ours: $\text{rank}(i) = Q_\theta\big((x_S, 0),\ \text{add } i\big)$

C. DQN: state, actions, reward, training

- **State** $s = (x_S, g)$: $x_S \in \{0,1\}^m$ marks the chosen items, $g = (P_{best} - P(S))/\bar{p}$ is the gap to the episode's best ($\bar{p}$: mean item profit).
- **Actions**: add i or remove i ($2m$ outputs). Infeasible moves are masked; undoing a recent move is tabu for 30 steps.
- **Reward**: only new bests count, $r_t = \max\big(0,\, P(S_{t+1}) - P_{best}\big)/\bar{p}$, so an episode's return equals its total improvement of the best.

Bellman target

$$y = r + \gamma \max_{a' \in \mathcal{A}(s')} Q(s', a')$$

Q-learning update (table)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s', a') - Q(s,a) \,\big]$$

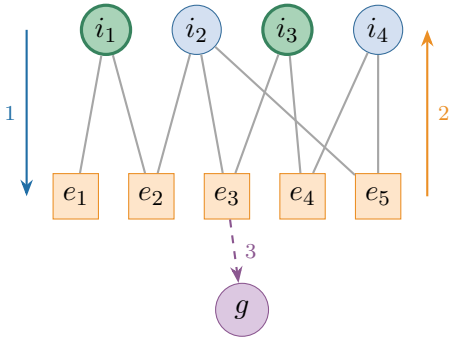
DQN loss (network, y from $Q_{\bar{\theta}}$)

$$\mathcal{L}(\theta) = \big(Q_\theta(s,a) - y\big)^2$$

- 1000 episodes of 100 steps; replay buffer 10^4, batch 128.
- Target network $Q_{\bar{\theta}}$ synced every 5 episodes; $\gamma = 0.99$; Adam, learning rate 10^{-3}.
- Exploration: ε-greedy ($\varepsilon_0 = 1$, $\times 0.99$ per episode), otherwise sample from softmax(Q/τ), $\tau = 5$.

[8] V. Mnih *et al.* (2015)

D. Graph neural network: message passing



Items (circles, green = chosen), elements (squares), global node g. One round: (1) items to elements, (2) elements to items, (3) readout to g.

Inputs: items h_i^I (chosen, profit, AW to add / remove, fits, efficiency), elements h_j^E (weight, covered, covered once, $c_j/\deg j$), global g (W/C, residual capacity, gap, $|S|/m$). Then three rounds of

$$
\begin{aligned}
h_j^E &\leftarrow \mathrm{LN}\big(h_j^E + \phi_E[\,h_j^E,\ \mathrm{mean}_{i\ni j}\, h_i^I,\ \mathrm{mean}_{i\in S,\, i\ni j}\, h_i^I\,]\big)\\
h_i^I &\leftarrow \mathrm{LN}\big(h_i^I + \phi_I[\,h_i^I,\ \mathrm{mean}_{j\in U_i}\, h_j^E,\ g\,]\big)\\
g &\leftarrow \mathrm{LN}\big(g + \phi_G[\,g,\ \mathrm{mean}_i\, h_i^I,\ \mathrm{mean}_j\, h_j^E\,]\big)
\end{aligned}
$$

Heads shared by all items: $Q(s, \text{add } i) = \psi_{add}[h_i^I, g]$, $Q(s, \text{remove } i) = \psi_{rem}[h_i^I, g]$, so the number of parameters does not depend on m or n.

[9] J. Gilmer *et al.* (2017)

IV. RESULTS

A. Profit per instance: Set I

$\alpha = 0.10,\ \beta = 0.75$						
$m \times n$	BABC	QL	gbPSO	Best†	**Ours**	Δ
85×100	11664	11619	**12045**	**12045**	**12045**	0
100×85	13251	13251	**13283**	**13283**	**13283**	0
100×100	13860	**14044**	**14044**	**14044**	**14044**	0
185×200	13047	13503	**13696**	**13696**	**13696**	0
200×185	13241	13405	13405	**13521**	**13521**	0
200×200	11846	12196	**12522**	**12522**	**12522**	0
285×300	11158	11395	**11568**	**11568**	**11568**	0
300×285	10428	10952	11335	**11563**	**11563**	0
300×300	12186	12644	12695	**12817**	12784	−33
385×400	10085	10022	10483	**10600**	10490	−110
400×385	10766	11242	**11484**	**11484**	**11484**	0
400×400	10626	11158	11432	**11665**	**11665**	0
485×500	10823	10823	11097	**11321**	11119	−202
500×485	10784	11414	**11771**	**11771**	11722	−49
500×500	10755	10888	10991	**11249**	11245	−4

$\alpha = 0.15,\ \beta = 0.85$						
$m \times n$	BABC	QL	gbPSO	Best†	**Ours**	Δ
85×100	**12369**	**12369**	**12369**	**12369**	**12369**	0
100×85	12238	**12479**	12348	**12479**	**12479**	0
100×100	**13508**	**13508**	**13508**	**13508**	**13508**	0
185×200	10602	10973	**11298**	**11298**	**11298**	0
200×185	13829	14044	**14215**	**14215**	**14215**	0
200×200	11521	11911	**12317**	**12317**	12113	−204
285×300	10528	11377	11763	**11802**	11714	−88
300×285	12012	12245	12245	**12607**	12500	−107
300×300	10382	11282	11425	**11585**	**11585**	0
385×400	9456	9738	10302	**10506**	10490	−16
400×385	9649	10419	10526	**11209**	10758	−451
400×400	9541	10520	10927	**11325**	**11325**	0
485×500	9333	9649	10117	**10220**	10120	−100
500×485	9090	9729	10022	**10238**	9938	−300
500×500	9318	9938	10194	**10381**	10186	−195

†I2PLS = KBTS = E2LS = AES-BM on all of Set I. Bold: best in the row. Δ = Ours − best. Ours is best on 17/30.

Data: `results_csv/profit.csv`. BABC [3], QL and gbPSO [10], I2PLS [11], KBTS [4], E2LS [5], AES-BM [12]

IV. RESULTS

B. Profit per instance: Set II

$\alpha = 0.10,\ \beta = 0.75$				
$m \times n$	KBTS	E2LS	**Ours**	Δ
585×600	**10393**	**10393**	10198	−195
600×585	**9914**	**9914**	9688	−226
600×600	**10524**	**10524**	10391	−133
685×700	**10121**	**10121**	10103	−18
700×685	**9881**	**9881**	9781	−100
700×700	**9786**	**9786**	9709	−77
785×800	**9384**	**9384**	9192	−192
800×785	9837	**9937**	9698	−239
800×800	**9932**	**9932**	9787	−145
885×900	**9318**	**9318**	**9318**	0
900×885	**9725**	**9725**	9594	−131
900×900	**9745**	**9745**	9648	−97
985×1000	9193	**9234**	9101	−133
1000×985	9668	**9689**	**9689**	0
1000×1000	9544	**9551**	9433	−118

$\alpha = 0.15,\ \beta = 0.85$				
$m \times n$	KBTS	E2LS	**Ours**	Δ
585×600	**9256**	**9256**	9119	−137
600×585	**9357**	**9357**	9252	−105
600×600	**9062**	**9062**	8893	−169
685×700	**9176**	**9176**	9064	−112
700×685	**9163**	**9163**	9051	−112
700×700	**9229**	**9229**	9154	−75
785×800	**8746**	**8746**	8505	−241
800×785	**9024**	**9024**	8898	−126
800×800	**9101**	**9101**	8831	−270
885×900	**8425**	**8425**	**8425**	0
900×885	**8620**	**8620**	8342	−278
900×900	**8990**	**8990**	8965	−25
985×1000	8528	**8612**	8426	−186
1000×985	8453	**8455**	8344	−111
1000×1000	8474	**8538**	8248	−290

Only KBTS and E2LS report Set II. Bold: best in the row. Δ = Ours − best. Ours is best on 3/30.

Data: `results_csv/profit.csv`. KBTS [4], E2LS [5]

V. REFERENCES

[1] Olivier Goldschmidt, David Nehme, and Gang Yu. Note: On the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.

[2] Ashwin Arulselvan. A note on the set union knapsack problem. *Discrete Applied Mathematics*, 169:214–218, 2014.

[3] Yichao He, Haoran Xie, Tak-Lam Wong, and Xizhao Wang. A novel binary artificial bee colony algorithm for the set-union knapsack problem. *Future Generation Computer Systems*, 78:77–86, 2018.

[4] Zequn Wei and Jin-Kao Hao. Kernel based tabu search for the set-union knapsack problem. *Expert Systems with Applications*, 165:113802, 2021.

[5] Wenli Zhu and Liangqing Luo. Efficient and effective local search for the set-union knapsack problem and budgeted maximum coverage problem. *arXiv preprint arXiv:2207.00749*, 2022.

[6] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[7] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

[10] Fehmi Burcin Ozsoydan and İlker Gölcük. A reinforcement learning based computational intelligence approach for binary optimization problems: The case of the set-union knapsack problem. *Engineering Applications of Artificial Intelligence*, 118:105688, 2023.

[11] Zequn Wei and Jin-Kao Hao. Iterated two-phase local search for the set-union knapsack problem. *Future Generation Computer Systems*, 101:1005–1017, 2019.

[12] Juntao Zhao and Mhand Hifi. An adaptive evolutionary search-based method for efficiently tackling the set-union knapsack problem. *Information Sciences*, 676:120855, 2024.